Introduction to Blockchain

CSE-355

Ahmed Akhtar
Department of Computer Science
SMCS, IBA Karachi
Spring 2026

Lecture 03
Tuesday, Jan 27, 2026

Building consensus: step by step

The effort to synchronize all individual journals results in the LEDGER

1

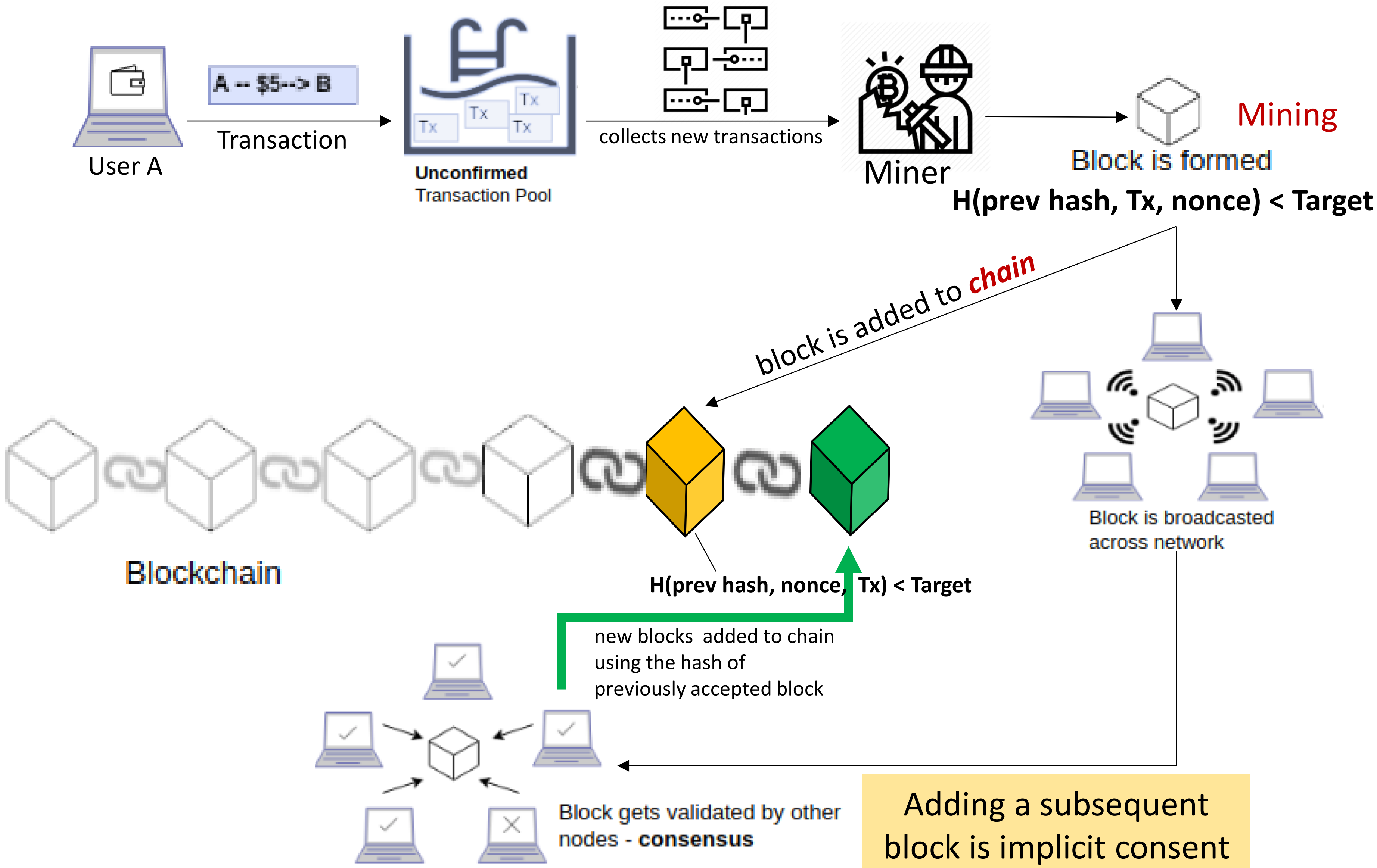
UNCONFIRMED TRANSACTIONS

End users create transactions and broadcast them to the network though various means. These transactions are waiting for confirmation.

End users (via nodes, SPV, API)

PROCESSING

How is a block added to the blockchain?

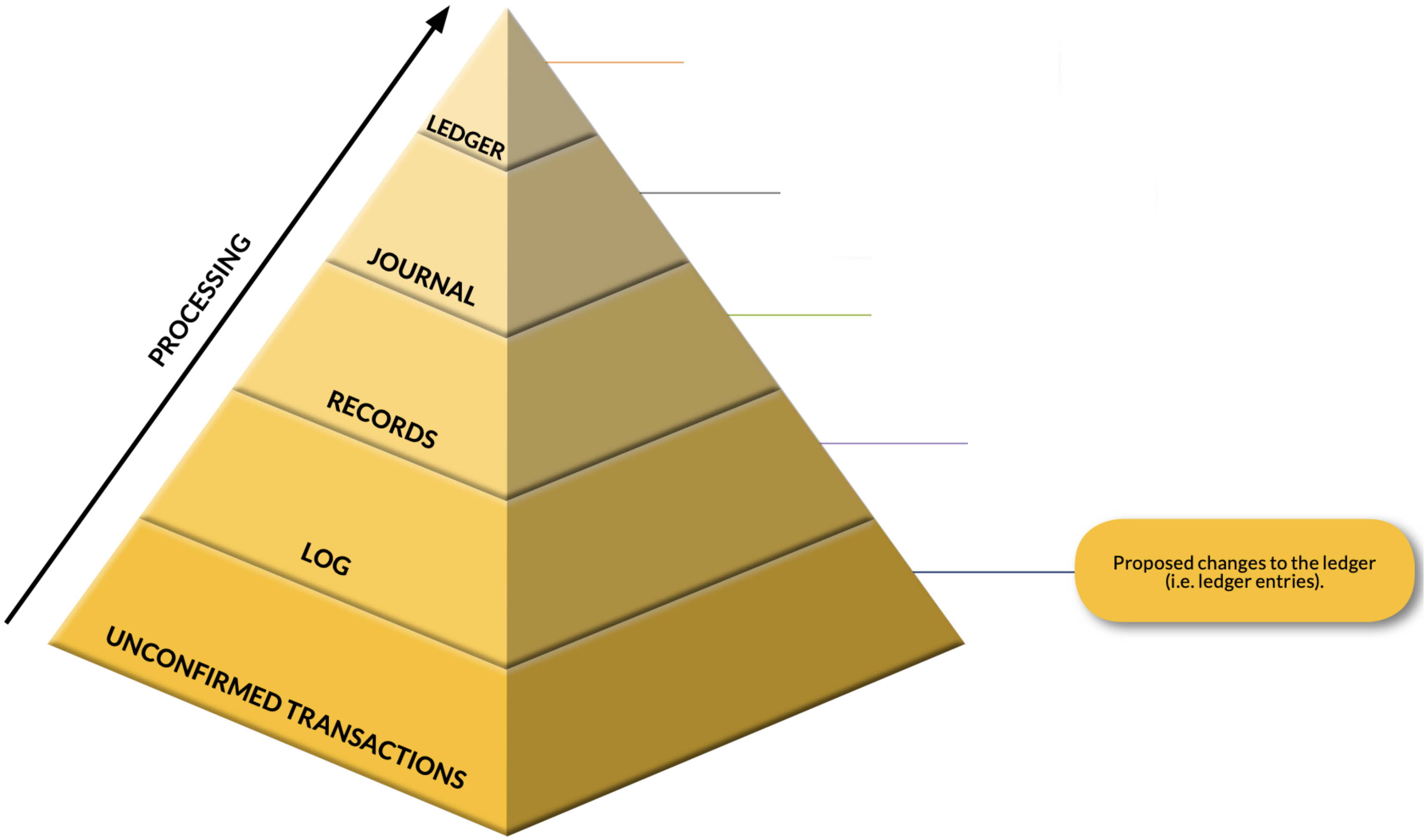


Adding a subsequent block is implicit consent

Data Lifecycle: Stages to Consensus

- Transaction – initiated and propagated by ***end users***
  - Any proposed change to the ledger (broadcasted to the network)
- LOG – created by ***auditors*** (or ***validating nodes***)
  - Set of valid yet unconfirmed transactions put in LOG
  - Each auditor has their own log/mempool
- Record – created by special nodes (***miners*** or ***record producers***)
  - Arbitrarily select a set of transactions from their log
  - Put the selected transactions into a candidate record and broadcast that candidate record to all nodes they are connected to
- Journal – locally created by ***each node*** that receives the record
  - If the candidate record complies with protocol rules
- Ledger emerges from the set of journals

Recap: building consensus



Characteristics of an ideal blockchain / DLT

- Shared recordkeeping
  - Multiple entities writing data
- Multi-party consensus
  - All parties must come to an agreement
- Independent validation
  - Every auditor (fully validating node) should be able to recompute (and thus verify) the entire state of the system
- Tamper evidence
  - Participants can trivially detect non-consensual **changes** to records
  - Example of such changes? How detected?
- Tamper resistance
  - Hard for a single entity to tamper past records (transaction history)

History: Byzantine Generals Problem

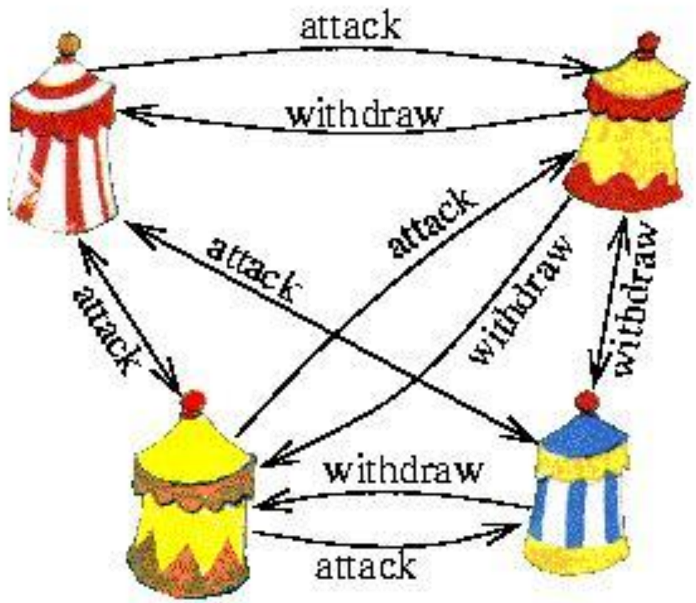
Several divisions of the Byzantine army

Camped outside an enemy city

Each division commanded by its own general

Generals must agree upon

A common battle plan



Some of them may be traitors – will prevent loyal generals from reaching an agreement

History: Byzantine Generals Problem

- Battalions of Byzantine army, each led by their own General
- If Generals attack collectively: they will win, else will lose (likely)
- Need: consensus between the Generals (multi-party consensus)
- What may hinder this? (the challenges)
- The Messengers sent among the Generals, may:
  - Be intercepted by the enemy
  - Only reach some generals but not all
  - Accidentally relay incorrect message
  - Forge a message maliciously
  - Be told a wrong message by a malicious General
- Consensus building is hard. What options do we have?

How this situation relates to computing systems?

- Decentralized computing systems may encounter ***byzantine faults***
  - Broken hardware
  - Congested network
  - Disconnected nodes
  - Incorrect processing of requests
  - Corrupted local states
  - Nodes producing inconsistent output
  - A set of nodes launching a malicious attack
- Similar issues as faced by the Byzantine Generals Problem
- Byzantine fault tolerance
  - A measure of the capability to prevent/tolerate byzantine failures
- Bitcoin blockchain: ***first*** to solve for ***unknown*** number of Generals

Techniques used in Bitcoin Blockchain

- Public-key cryptography
  - Used for unforgeable digital signatures from authenticated sources
  - ***Authenticates*** Generals and ***Prevents*** messengers from forging

- Atomic broadcast of all transactions
  - No General missed from receiving the transaction

- Proof of Work (PoW)
  - Malicious Generals will have to pay a hefty price for deceiving

- Tampering of a block becomes harder and harder – **Chaining**
  - Every new block further ***cements*** the past transactions

Blockchain: How does it get built up?

- The longest chain is picked
  - Is there a situation when one has to “pick” from a choice of chains?

- Forks
  - Temporarily different views of journals

- Finality
  - A transaction becomes “final” only after multiple confirmations
  - Why?
  - Since forks can temporarily reorder or exclude it

Types of blockchain

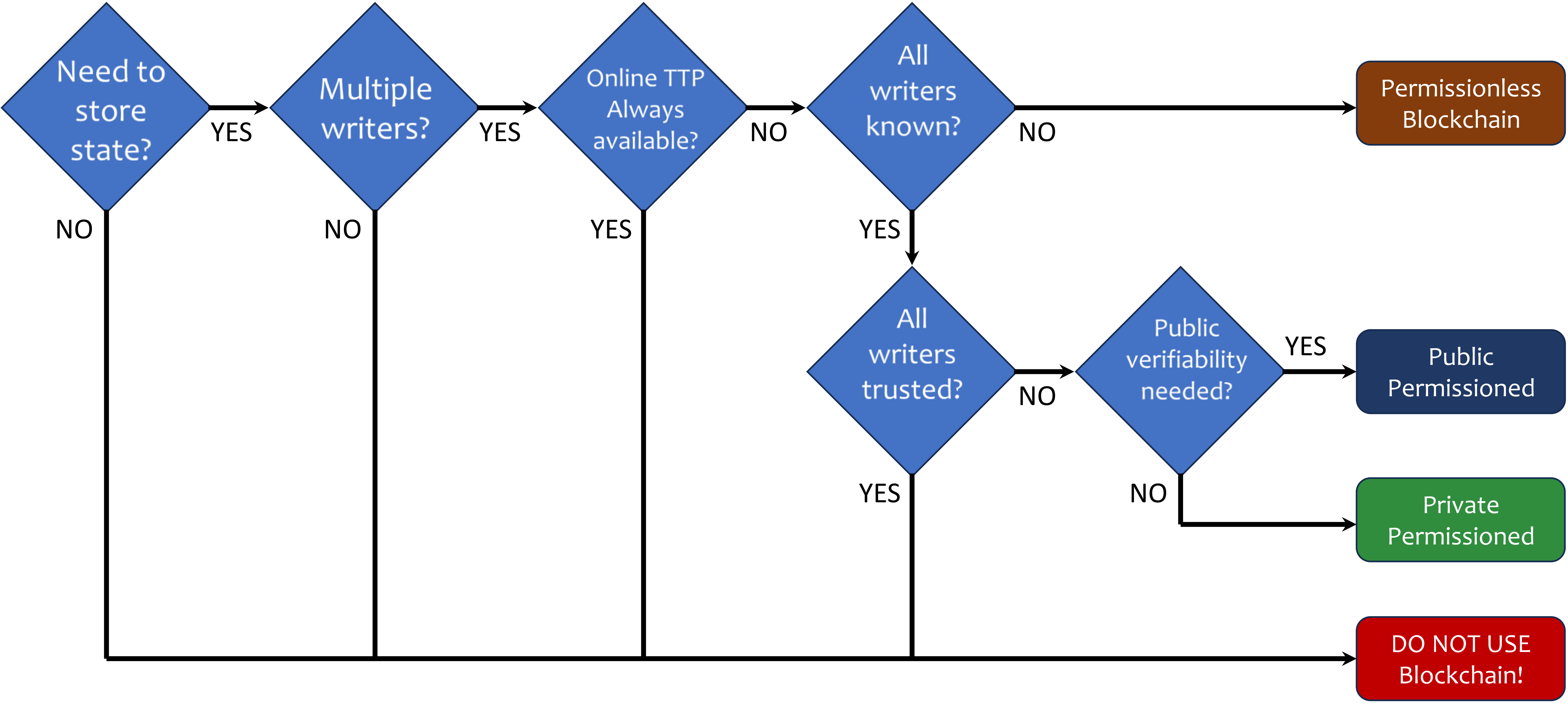
- Permissionless (must be public)
  - Examples: Bitcoin and Ethereum
  - Anyone can join/leave at will at any time
  - No central entity managing the membership
  - Globally readable (anyone can be a reader)

- What about privacy and anonymity?
  - Privacy? Compromised by design (need auditability)
  - Anonymity? Feasible with cryptographic procedures

Permissionless vs. Permissioned

	Public	Private
Permissionless		
Permissioned		

When (not) to use a Blockchain?



Cryptocurrencies: A blockchain use case

- The relation between Bitcoin and Blockchain?
- What do miners "mine"? – (Is it like GOLD mining?)
  - Miners mine (i.e., **create** and **add**) blocks in a blockchain
- What is a bitcoin (BTC), then?
  - It is a digital (crypto)currency – tradeable on its own blockchain
  - Via transactions recorded on that blockchain
- The Bitcoin blockchain ***rewards*** miners with Bitcoins
  - When they create a new block that is accepted via consensus
- Where do those new Bitcoins come from? – the protocol
  - The system simply writes a transaction on the blockchain

Lecture 04
Thursday, Jan 29, 2026

Bitcoin Transaction: Step by Step

- User initiates the transaction
  - Typically, from their wallet or crypto exchange application

- Transaction broadcast for “execution”

- Placed in a pool of “unconfirmed transactions”
  - Multiple local pools rather than one giant pool
- Waiting to be picked up by “Miners” / nodes

- Why will miners pick up? Incentive?
- Reward for mining (Subsidy rewards + Transaction fees)

Bitcoin Transaction: Step by Step

- Miner(s) pick up unconfirmed transaction(s)
  - Many-to-many paradigm
    - One transaction may be picked by multiple miners
    - A miner may pick multiple transactions to put those in one “block” (max ~1MB on the Bitcoin blockchain)
  - Blocks are created and stored by the miners/nodes

- Miners solve the block for an eligible hash (candidate record)
  - Search over the nonce space
  - An eligible hash (hard to find, easy to verify)
  - A correct hash is the Proof of work

Bitcoin Transaction: Step by Step

- Broadcasting the solution
  - A successful miner broadcasts the solution (block+hash)
- Signature verification
  - Other miners verify the hash (easy!)
  - Consensus algorithm to agree
  - Block added to the blockchain (to their own journal, in fact)
- Confirmation
  - Additional blocks added after this one
  - Subsidy rewards – available to spend after 100 confirmations
- On receiving a block, Miners must ***start afresh*** on a ***new block*** using the hash of the latest-found block as part of the data. Why?

Bitcoin: Ownership and Transactions

- The cryptocurrency (BTC) is always ***locked***
  - **To the owner**

- **Transaction**
  - payer ***unlocks*** (only they can!) and
  - then ***locks*** again for the payee (only they will be able to unlock!)

- Minting of new BTC
  - Happens only at block creation (and at no other times)
  - Gets locked to the creator of the block

BTC and Fiat

- How did BTC get related to Fiat currency?

- Historical Fact: The New Liberty Standard Exchange recorded the first exchange of BTC for fiat (dollars) in late 2009

- What was the transaction?
  - Users (on the BitcoinTalk forum) traded 5,050 bitcoins
- For how much?
  - Total value: $5.02 (sent via PayPal) @ $0.00099 per BTC
  - The average per BTC price last month ~ $89,000 (Jan 2026)

Understanding Blockchains

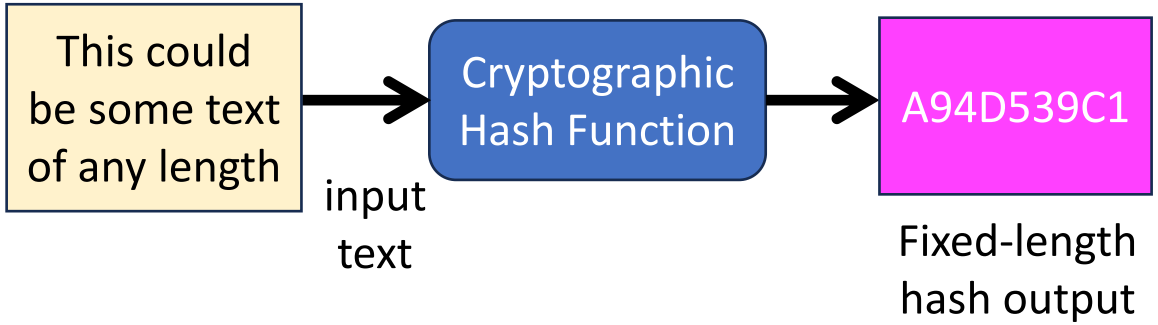
Cryptographic Hashing

Digital Signatures

Consensus

Hash Function: Properties

- Input: **arbitrary size** plain text
- Output: **fixed size** message digest
- Output is "**random**" looking
  - Not random (actually, deterministic)
  - About half ones and half zeros
- **Avalanche** effect
  - Change one bit in the input; about half the output bits change
- **Preimage** resistance
  - Given Y, can't find X such that Hash(X) = Y
- **Collision** resistance
  - Can't find X, Z such that X ≠ Z and Hash(X) = Hash(Z)

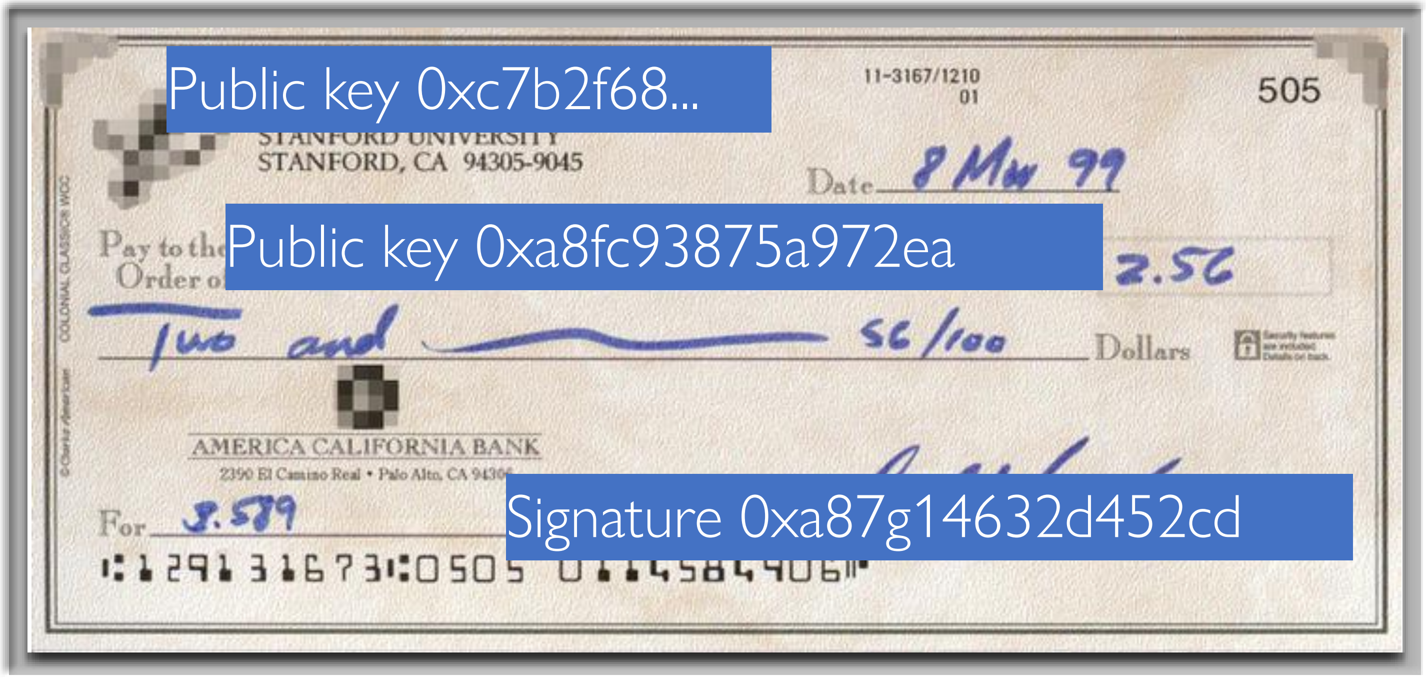


- Online tools or your OS can **quickly compute** Hash functions

What is needed for a transaction?

Compare with a check

- **Payer**
- **Payee**
- **Amount**
- **Authorization**



- Can this check be honored?
- Does the account have this much?
- Account balance is checked offline

What is needed for a blockchain transaction?

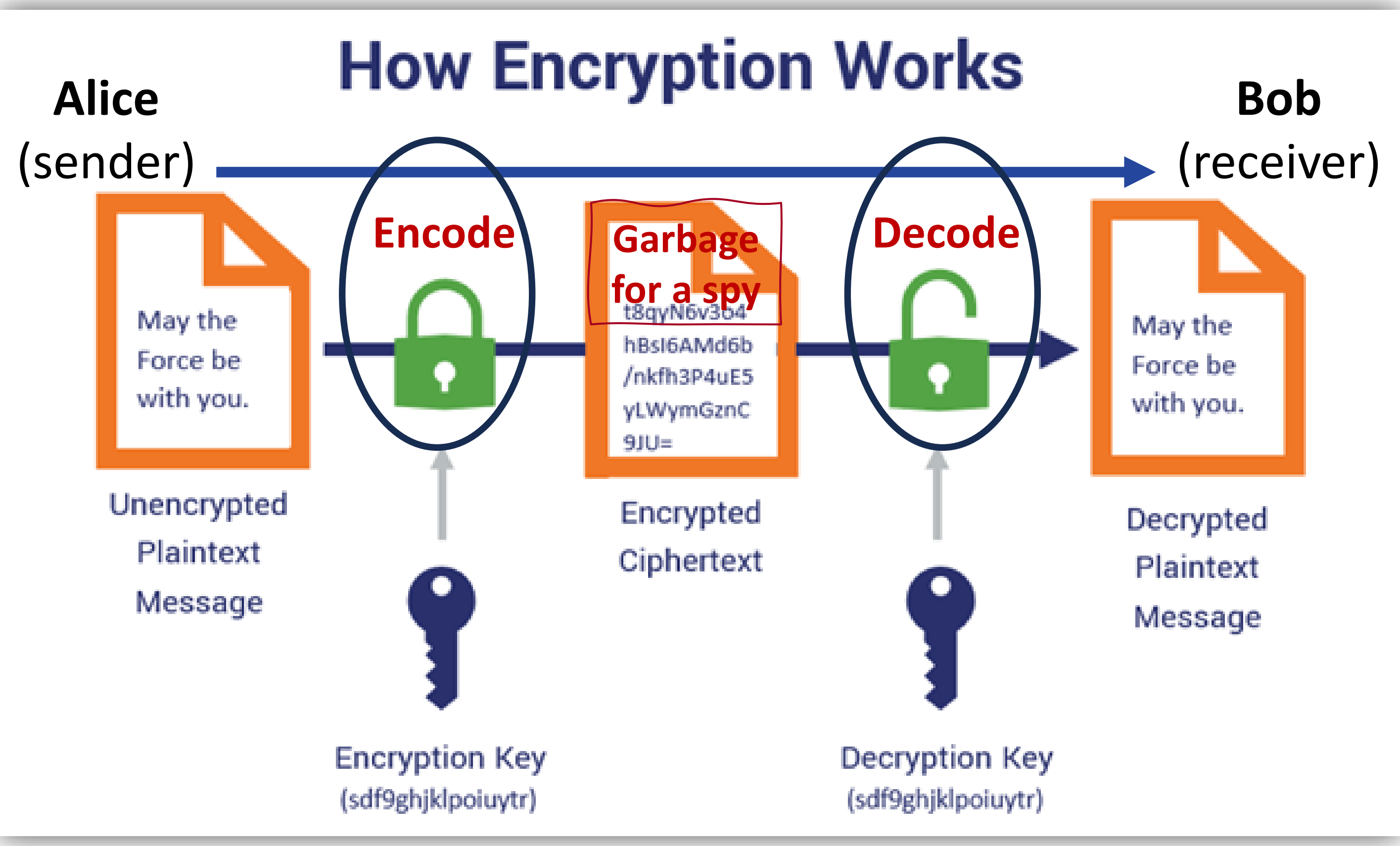
Digital Signatures

- What are they?
  - Like manual signatures, but different for each occurrence
- Why are they needed [in blockchains]?
  - *Authenticate* the **sender** (end user who broadcasts transactions)
  - *Avoid forgery* in the **message** sent to the receiver (**transactions**)
- In a transaction, we also need to:
  - *Identify* the **receiver** (or at least the address of the receiver)

- How do digital signatures work?
  - Using public key cryptography infrastructure

What is Cryptography?

GOAL: Send message that a spy is unable to make sense of!



Symmetric: same **shared** key for encryption and for decryption

Symmetric Key Cryptography
Big Challenge:

key sharing

Thought Question:
If Bob receives some data

Can he be sure it came from Alice?

Public Key Infrastructure and Digital Signatures

- Governed by ***three*** functions

- **Generate Keys()** – a publicKey, privateKey pair is generated
  - A key is nothing more than a sequence of bits

- **Sign(message, privateKey)** – sender will do it
  - Function output is the signature
  - Signature ***along with message*** is sent (sent where?)

- **Verify(message, signature, publicKey)** – receiver will do it
  - Function output is Boolean (Verified or Not)